

Bilgisayar Virüsleri ve Güvenlik Modelleri

CEN429 Güvenli Programlama — Hafta 2

Dr. Öğr. Üyesi Uğur CORUH · 25.09.2026

0. Temel kavramlar (sıfırdan)

Neden bu bölüm?

Bu hafta "polimorfik virüs", "Bell–LaPadula", "CVSS" gibi terimler geçecek.

Hiçbirini bilmediğinizi varsayıyoruz.

Önce hepsini **tek tek** tanımlayalım.

Zararlı yazılım (malware)

- **Zararlı yazılım:** zarar vermek için yazılmış program.
- "Virüs" bir alt türdür; hepsi virüs değildir.
- Türler: virüs, solucan, truva atı, fidye, casus...

Virüs, solucan, truva atı

- **Virüs:** başka bir programa **bulaşarak** yayılır.
- **Solucan:** kendi başına, ağ üzerinden **kendini kopyalar**.
- **Truva atı:** faydalı görünüp gizli zararlı taşır.

Fidye ve diğerleri

- **Fidye (ransomware):** dosyaları şifreleyip fidye ister.
- **Casus (spyware):** gizlice bilgi toplar.
- **Arka kapı (backdoor):** gizli erişim bırakır.

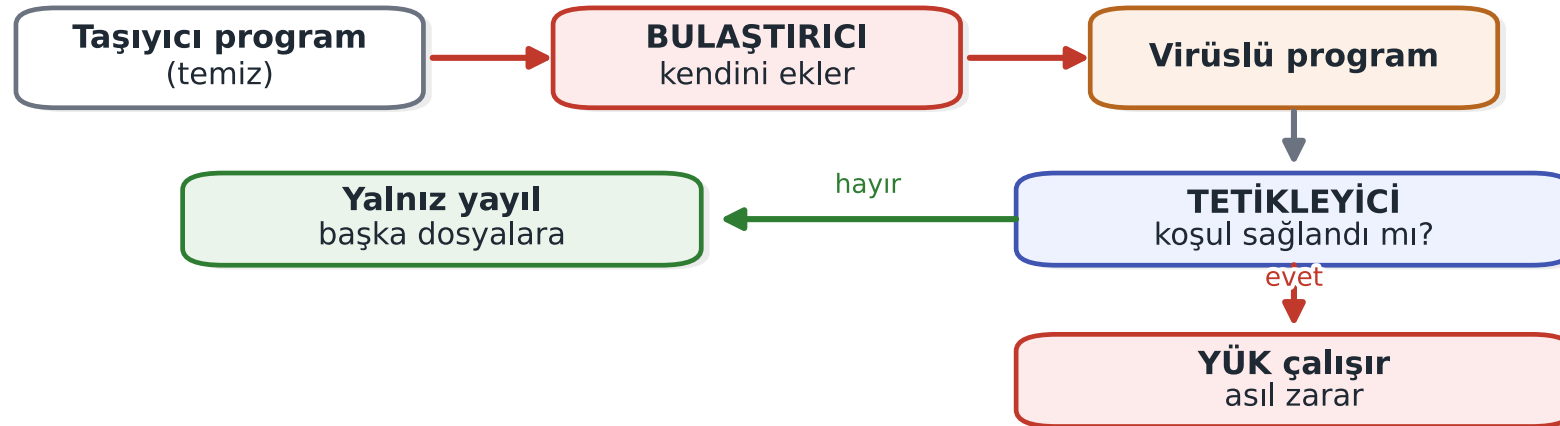
Bir virüsün üç parçası

- **Bulaştırıcı:** nasıl yayılır.
- **Tetikleyici:** ne zaman etkinleşir.
- **Yük (payload):** ne yapar.

Virüsün üç parçası — şema

Bir virüsün üç parçası

bulaştırıcı · tetikleyici · yük



'Ne zaman?' sorusuna TETİKLEYİCİ bakar; koşula bağlı patlayana mantık bombası denir.

Gizlenme: polimorfik/metamorfik

- **Polimorfik:** her kopyada kendini **farklı şifreler** (imzadan kaçır).
- **Metamorfik:** her kopyada kodunu **yeniden yazar**.
- Amaç: imza tabanlı tespitten kaçmak.

Tespit: imza vs sezgisel

- **İmza tabanlı:** bilinen zararlının parmak izini arar (hızlı, yeniyi kaçırır).
- **Sezgisel/davranış:** şüpheli **davranışı** arar (yeniyi bulur, yanlış alarm).

Entropi (rastgelelik)

- **Entropi:** verinin ne kadar rastgele görüldüğü.
- Şifreli/paketlenmiş kod **yüksek entropili**.
- Tespitte ipucu: yüksek entropili bölge şüphelidir.

Erişim denetimi

- **Erişim denetimi:** kim (özne), neye (nesne), ne yapabilir (hak)?
- Bir **matris** ile modellenir.
- Modeller: DAC, MAC, RBAC.

DAC / MAC / RBAC

- **DAC:** sahibi izinleri belirler (Unix dosya izinleri).
- **MAC:** sistem zorunlu kurallar koyar (etiketli).
- **RBAC:** izinler **rollere** bağlı.

Biçimsel modeller

- **Bell–LaPadula: gizlilik** (yukarı okuma yok).
- **Biba: bütünlük** (aşağı okuma yok — BLP'nin tersi).
- **Clark–Wilson:** ticari bütünlük (iyi biçimli işlemler).

Denetim kaydı (audit log)

- **Denetim kaydı:** kim ne zaman ne yaptı — **kanıt** olarak günlük.
- Kurcalamaya dayanıklı olmalı.
- Günlük enjeksiyonuna (CWE-117) dikkat.

CWE, CVE, CVSS

- **CWE:** zayıflık **türü** kataloğu (ör. CWE-416).
- **CVE:** **belirli** bir ürünlerdeki açık (ör. CVE-2024-xxxx).
- **CVSS:** bir açığın **ciddiyet puanı** (0–10).

OWASP ve zafiyet yaşam döngüsü

- **OWASP Top 10 / MASVS:** yaygın açıklar ve mobil gereksinimler.
- **Sorumlu ifşa:** açığı önce üreticiye bildirme.
- Açık: keşif → bildirim → yama → yayın.

Zafiyet yaşam döngüsü — şema

Zafiyet yaşam döngüsü ve sorumlu ifşa

doğru yol: önce satıcıya bildir, yama çıksın, sonra açıkla



Kötüye giderse: yama yokken saldırı = ZERO-DAY. Bu yüzden ifşa sırası önemlidir.

Şimdi hazırız

Terimler:

zararlı yazılım · virüs/solucan/truva/fidye · virüsün üç parçası · polimorfik · imza/sezgisel tespit · entropi · erişim denetimi (DAC/MAC/RBAC) · BLP/Biba · denetim kaydı · CWE/CVE/CVSS · OWASP

Şimdi: tehdit, model, sınıflandırmanın büyük resmi.

Bugünün planı (3 saat)

Saat	Konu
1	Zararlı yazılım: tarih, anatomi, türler · gizlenme
2	Demo 1–2 · karşı önlemler · saldırı ağacı (Demo 4) · erişim ve modeller (Demo 3, 6)
3	CWE · OWASP · CVE · CVSS (Demo 5) · yaşam döngüsü · proje

Demolar: `code/week-02` — Windows `.\demo.ps1` · WSL/Linux `sh demo.sh`

Kısa tarihçe — zararlı yazılım ve güvenlik modelleri

- **1949** — von Neumann: **kendini çoğaltan** otomata (virüsün matematiksel kökü)
- **1971** Creeper · **1986** Brain (ilk PC virüsü) · **1988 Morris Worm** interneti durdurur
- **1973–77** — **Bell–LaPadula** (gizlilik), **Biba** (bütünlük); **1987** Clark–Wilson
- **1999 → 2006** — **CVE** · **CWE** · **CVSS**: ortak sınıflandırma dili

İki ayrı kol: **zararlıyı tanıma** + **erişimi modelleme**. Bugün ikisini birden görüyoruz.

Öğrenme çıktısı ve kapsam

- **ÖÇ.1:** Yaygın yazılım güvenlik açıklarını **tanımlar ve sınıflandırır**.
- Bu hafta üç konu bir arada:
 - **Tehdit:** zararlı yazılım türleri ve gizlenmesi
 - **Savunma:** erişim denetimi ve biçimsel modeller
 - **Ortak dil:** CWE · CVE · CVSS · OWASP · MASVS

 **Etik:** Hiçbir demoda gerçek zararlı yazılım yok. Hepsi kendi klasöründe, yönetici yetkisi olmadan çalışan **güvenli benzetimlerdir**.

Zararlı yazılım: türler ve tarih

"Virüs" her şey değildir

Zararlı yazılım (malware): sahibinin izni dışında zarar veren her program.

- Virüs, zararlı yazılımın **yalnızca bir türüdür.**
- Ayrım **nasıl yayıldığına** göre yapılır:
 - **Virüs:** başka dosyaya iliştilir, taşıyıcı çalışınca yayılır
 - **Solucan:** kendi başına, ağ üzerinden yayılır
 - **Truva atı:** masum görünür, kendini kopyalamaz
 - **Fidye / rootkit / bot / silici:** yük ya da davranış türleri

Haftanın üç parçası — şema

Bu haftanın üç parçası

tehdidi tanı → savun → ortak dille adlandır



Üçü birlikte: ne olduğunu bilmeden savunamazsın, adlandırmadan yönetemezsin.

Kısa tarih: dönüm noktaları

Yıl	Olay	Türü
1984	Cohen'in tanımı	Kuram: "tam tespit imkânsız"
1986	Brain	İlk yaygın PC virüsü (önyükleme sektörü)
1988	Morris	İnternette yayılan ilk büyük solucan
2000	ILOVEYOU	E-posta ekiyle milyonlar
2001	Code Red	Ağ açığıyla bellekte yayılan solucan
2010	Stuxnet	Endüstriyel denetleyiciyi hedef alan solucan
2017	WannaCry	Kendi kendine yayılan fidye yazılımı
2017	NotPetya	Fidye gibi görünen silici (yıkıcı)

Cohen 1984: tespitin sınırı

Bir virüs, kendini (belki değiştirerek) başka programlara **kopyalayabilen** programdır.

- Cohen kanıtladı: "**Bu program virüs mü?**" sorusunu her durumda doğru yanıtlayan genel bir algoritma **yoktur** (durma probleminden indirgenir).
- Pratik sonuç: hiçbir antivirüs %100 tespit edemez.
 -  **Yanlış pozitif:** temiz dosyayı yakalar
 -  **Yanlış negatif:** zararlıyı kaçırır

Bunu Demo 01'de kendi gözümüzle göreceğiz.

Bir virüsün üç parçası

Parça	İşi	Hastalık benzetmesi
Bulaştırıcı	Kendini kopyalar	Hücreye girip çoğalma
Tetikleyici	"Ne zaman?" koşulu	Kuluçka süresi
Yük	Asıl zarar	Semptomlar

- Tetikleyici koşulu bekleyip patlayan koda **mantık bombası** denir.
- Her parça zorunlu değildir: yalnız yayılan, yüksüz bir virüs de vardır (yine de zafiyettir).

Virüs alt türleri

- **Program (dosya) virüsü:** çalıştırılabilir dosyaya bulaşır; dosya çalışınca virüs de çalışır.
- **Makro virüsü:** ofis belgesinin makro diline yazılır; belge açılınca çalışır (Melissa, 1999).
- **Önyükleme sektörü virüsü:** diskin ilk sektörüne bulaşır, işletim sisteminden **önce** çalışır (Brain, 1986).
 - Modern **UEFI Secure Boot** bu sınıfı büyük ölçüde zorlaştırdı.

Solucan, truva atı, fidye

Tür	Yayılma	Örnek
Solucan	Kendi başına, ağdan; kullanıcı gerekmez	Morris, Code Red, WannaCry
Truva atı	Masum görünür; kullanıcı çalıştırır; kopyalanmaz	"Bedava oyun", sahte güncelleme
Fidye yazılımı	Dosyaları şifreler, para ister	CryptoLocker, WannaCry

Solucanın tehlikesi hızdır: dakikalar içinde yüz binlerce makine.

Diğer türler

- **Rootkit:** kendini ve diğer zararlıları işletim sisteminden **gizler** (dosya/süreç/ağ listesinden siler). Tespiti en zor sınıf.
- **Bot / botnet:** uzaktan komut alan makineler ağı (DDoS, spam, madencilik).
- **Casus / reklam yazılımı:** veri toplar ya da reklam gösterir.
- **Silici (wiper):** fidye gibi görünür ama veriyi **kalıcı yok eder** (NotPetya). Amaç para değil, yıkım.

Solucan ne kadar hızlı yayılır?

SI modeli — duyarlı (S) ve enfekte (I) makineler:

$$i(t+dt) = i(t) + \text{beta} * i(t) * (1 - i(t)/N) * dt$$

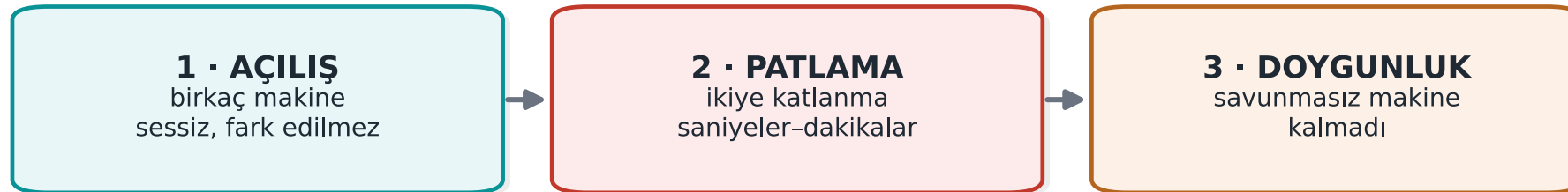
- $\text{beta} * i$: enfekte çoğaldıkça tarama hızlanır → **üstel** büyüme
- $(1 - i/N)$: duyarlı azaldıkça isabet düşer → **doygunluk**
- Sonuç: sessiz açılış → ani patlama → doygunluk (S eğrisi)

⚠ Savunmacının tek penceresi **açılış evresi**

Salgın modeli — şema

Solucan salgını: SI modeli

yayılma üstel — insan müdahalesi yetişemez



Morris (1988) ve Slammer (2003): dakikalar içinde internetin büyük kısmı etkilendi.

Demo 09 — Salgın simülasyonu

code/week-02/09-salgin-simulasyonu · yalnız hesaplama, ağ yok

Senaryo: Slammer benzeri (rastgele tarama, hızlı UDP)

54 s	78	0.1%	
1.5 dk	1430	1.9%	#.....
2.4 dk	45601	60.8%	#####.....
3.3 dk	74429	99.2%	#####

%90 doygunluk: Code Red ~13.3 sa · Slammer ~2.7 dk · Hitlist ~50 s

- Code Red (2001): ~359.000 sunucu · ikiye katlanma ~37 dk
- Slammer (2003): tek 404 baytlık UDP paketi · ~8,5 s'de ikiye katlanma
- ☒ Ders: insan hızında tepki yetişmez → **önceden yama, kapalı port, otomatik savunma**

Yayılma ve gizlenme

Gizlenme merdiveni

Yöntem	Ne değişir?	Savunma
Şifreleme	Gövde şifreli, çözücü sabit	Çözücüyü imzala
Oligomorfizm	Çözücünün birkaç biçimi	Hepsini imzala
Polimorfizm	Çözücü her kopyada yeniden üretilir	Emülasyonla çöz
Metamorfizm	Şifreleme yok, kodun tamamı yeniden yazılır	Davranışa bak

Anahtar fikir: şifreli gövde her kopyada farklı görünür, **çözülünce hep aynıdır.**

Şifreli / polimorfik virüsün yapısı

```
+-----+ +-----+
| Çözücü kod | --> | Şifreli gövde |
| (poli: her kopya |   | (her kopyada |
| farklı)      |   | farklı baytlar) |
+-----+ +-----+
              | çöz
              v
              +-----+
              | Açık gövde |
              | (HER ZAMAN AYNI) | --> imza burada tutar
              +-----+
```

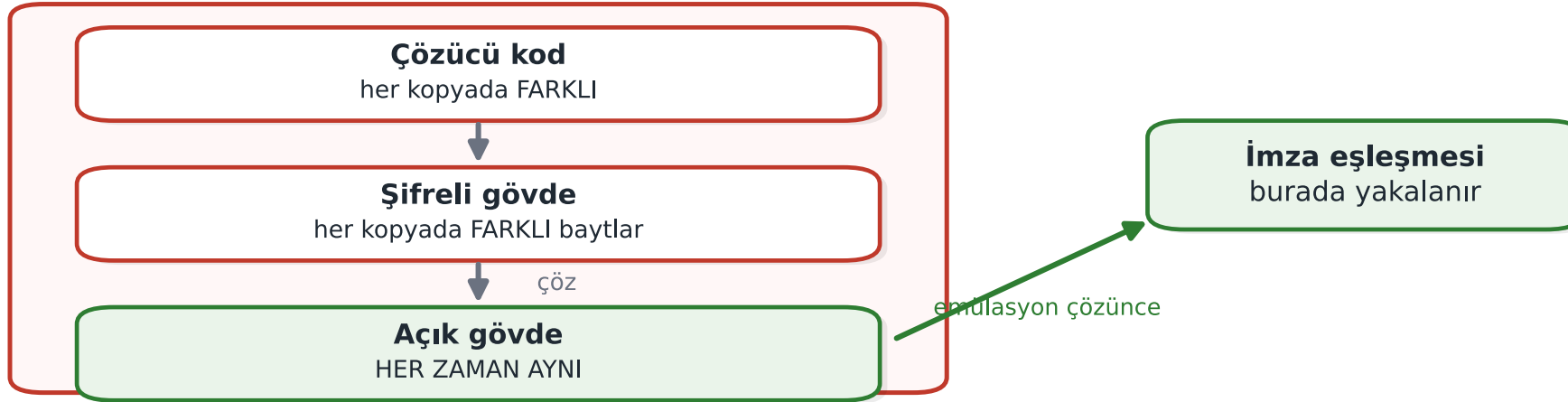
Emülasyon tam da bu son kutuyu görmek için çözücüyü çalıştırır.

Polimorfik virüs — şema

Polimorfik virüs neden imzayla yakalanmaz?

her kopyada baytlar değişir, açılan gövde aynı kalır

Şifreli / polimorfik virüs



Bu yüzden emülasyon kullanılır: virüsü yalıtılmış ortamda çalıştırıp AÇIK gövdeyi imzalar.

Demo 01 — Oyuncak antivirüs

code/week-02/01-imza-tarayici · Kitap: Tarif 12.1

- Zararsız, çalıştırılmayan bir "**KAPSÜL**" biçimi: başlık + çözücü + gövde.
- Dört yöntem, dört sınır:

Yöntem	Yakalar	Atlatılır
Hash	Birebir aynı dosya	Tek bayt değişince
Desen	Küçük değişiklik	Çözücü değişince (polimorfizm)
Sezgisel	İmzasız	Yanlış pozitif/negatif
Emülasyon	Hepsi	Anti-emülasyon

Demo 01 — Hash kırılğan, desen dayanıklı

ADIM 2 (hash):

```
ornek_a.bin          YAKALANDI  Ornek.MaviKedi.A  
ornek_a_1bayt.bin    TEMIZ      <- tek bayt degisti, ozet bambaska
```

ADIM 3 (desen):

```
poli_1.bin  YAKALANDI  Ornek.Cozucu.xs32  
poli_2.bin  YAKALANDI  Ornek.Cozucu.xs32  
poli_3.bin  TEMIZ      <- cozucu de degisti (polimorfizm)
```

Hash çığ etkisiyle tek baytta kırılır; **desen** çözücüyü yakalar ama polimorfizm onu da atlatır.

Demo 01 — Sezgisel: iki tür hata

ADIM 5 (sezgisel):

```
ornek_a.bin  TEMIZ      puan=  0   <- YANLIS NEGATIF (kacirdi)
poli_1.bin   SUPHELI    puan= 80
arsiv.bin    SUPHELI    puan= 50   <- YANLIS POZITIF (zararsiz)
```

- İmza olmadan polimorfiği yakaladı 
- Ama zararsız yüksek-entropili dosyayı şüpheli sandı 
- Ve düz gövdeli örneği kaçırdı 

Cohen'in belirsizliği tam burada.

Demo 01 — Emülasyon hepsini çözer

ADIM 6 (emulasyon):

```
poli_1.bin  YAKALANDI  ... | 5 komut, 0 BOS, xs32  
poli_2.bin  YAKALANDI  ... | 5 komut, 0 BOS, xs32  
poli_3.bin  YAKALANDI  ... | 8 komut, 4 BOS, lcg8
```

- Çözücüğü **güvenli sanal makinede** çalıştırır, açılan gövdeyi tarar.
- Üç polimorfik kopya da **aynı imzaya** döndü.
- Güçlü ama **pahalı**; zararlı "sanal makinedeyim" derse gizlenebilir.

Sonuç: tek yöntem yetmez → katmanlı savunma.

Entropi: rastgelelik ölçüsü

$$H = - \sum_{b=0}^{255} p(b) \log_2 p(b) \quad (0 \dots 8 \text{ bit/bayt})$$

- **H ~ 0:** hep aynı bayt · **H ~ 4–5:** metin/kod
- **H ~ 6:** derlenmiş program · **H ~ 8:** şifreli/sıkışık/rastgele

Antivirüs: "paketli/şifreli kod?" · EDR: "dosyaları hızla şifreleyen süreç?" (fidye)

Demo 02 — Entropi ölçer

code/week-02/02-entropi

tekduze.txt	0.00		tekduze
belge.txt	4.17	#####.....	dusuk
kod.bin	6.23	#####.....	orta
sifreli.bin	7.95	#####	YUKSEK
rastgele.bin	7.95	#####	YUKSEK

⚠ Entropi tek başına "zararlı mı?" demez: `.zip` / `.png` de yüksektir. **Beklenmedik yerde** yüksekse ipucudur.

Demo 02 — Sıcak bölge ve fidye izi

Karma dosya (pencere 512):

```
1024-1535  4.14  dusuk
1536-2047  7.56  YUKSEK  <- gizli sifreli bolum
2048-2559  7.66  YUKSEK
```

Fidye izi: belge.txt 4.17 --> sifreli.bin 7.95

- Paketlenmiş programın deseni: **küçük açıcı + yüksek entropili gövde**.
- Bir süreç kısa sürede çok dosyada bu sıçramayı yapıyorsa **davranış tabanlı** koruma alarm verir.

Karşı önlemler

Yöntem	Güçlü	Zayıf
İmza tabanlı	Hızlı, düşük yanlış pozitif	Bilinmeyen/polimorfiği kaçırır
Sezgisel	İmzasız yeni örnek	Yanlış pozitif/negatif
Davranış tabanlı	Gizlenmeye dirençli	Zarar başlamış olabilir
Kum havuzu	Gerçek davranışı görür	Yavaş; anti-VM
Emülasyon	Çözücüyü açar	Anti-emülasyon
İtibar / bulut	Hızlı yayılan tehdide iyi	Gizlilik, bağlantı

Modern EDR hepsini birleştirir, davranışa ağırlık verir.

Demo 07 — Kural tabanlı tespit

code/week-02/07-kural-motoru · YARA benzeri mini motor, **kendi zararsız desenlerimiz**

```
kural Kelime_Baglam {  
  dizge $a = "indir"  
  dizge $b = "calistir"  
  dizge $sihir = "KAPSUL/1"  
  kosul ($a and $b) and $sihir  
}
```

Adım	Sonuç	Ders
Zararsız kılavuz	Kelime_Cifti eşleşti	✗ Bağılamsız kural → yanlış pozitif
Değişmiş varyant	yalnız joker (??) desen yakaladı	✓ Değişmeyen iskeleti hedefle
Bozuk kural dosyası	REDDEDİLDİ	✓ Tespit aracı da bir ayrıştırıcıdır

Demo 08 — Bütünlük izleme (beyaz liste)

code/week-02/08-butunluk-izleme · SHA-256 manifesti + **HMAC mühür**

```
DEGISTI kutuphane.bin c603d7c6... -> 64b8f2b2... (49->65 B)
SILINDI veri.txt      (manifestte var, klasorde yok)
YENI     gizli.bin    (taban cizgisinde yok)
-- saldirgan manifesti yeniden yazar --
SONUC: butunluk korunmus    <- izleyici KANDIRILDI
MUHUR GECERSIZ -> manifeste GUVENME
```

✅ Özet değişikliği yakalar · ⚠️ **beklenen değerler de korunmalı** (HMAC/imza, anahtar ayrı yerde)

Tarif 12.2: CRC32 → SHA-256/HMAC · 6. haftada çalışma zamanı bütünlük denetimi

Olaylar: içeri nasıl girdiler?

Olay	Giriş yolu	Hata sınıfı
Morris (1988)	<code>fingerd</code> <code>gets()</code> taşması · <code>sendmail</code> DEBUG · <code>rsh</code> güveni	CWE-120/242 · 489 · 287
Code Red (2001)	IIS <code>.ida</code> taşması (CVE-2001-0500)	CWE-120
Slammer (2003)	SQL çözümleme hizmeti, tek UDP paketi (CVE-2002-0649)	CWE-121
Blaster (2003)	RPC/DCOM taşması (CVE-2003-0352)	CWE-120
Conficker (2008)	Server hizmeti yol işleme (CVE-2008-4250) · zayıf parola	CWE-119 · 521
WannaCry (2017)	SMBv1 (CVE-2017-0144) · yama 2 ay önce çıkmıştı	bellek güvenliği
NotPetya (2017)	güncelleme sunucusu üzerinden dağıtım	CWE-494
SUNBURST (2020)	derleme ortamına arka kapı	tedarik zinciri

Olaylardan programcıya dersler

1. **Sınır denetimi:** 1988–2008 salgınlarının çoğu tek bir arabellek taşması (1. ve 4. hafta)
2. **Sürümde hata ayıklama kodu bırakma** (Morris / `sendmail` DEBUG)
3. **Gereksiz hizmeti açma:** her port bir saldırı yüzeyi (Slammer, Blaster)
4. **İmzalı güncelleme + istemcide doğrulama** (NotPetya) — 3. ve 10. hafta
5. **Derleme ortamı ve tedarik zinciri güvenliği** (SUNBURST) — 5. hafta SBOM
6. **Güncelleme mekanizması güvenlik özelliğidir:** yama gecikmesi = açık kapı

Saldırı ağaçları

Saldırı ağacı — şema

Saldırı ağacı: başkasının hesabından ödeme

parantez içindeki sayı = saldırganın maliyeti (düşük = kolay)



En ucuz dal (3) önce kapatılır: savunma bütçesi en olası yola harcanır.

Saldırı ağacı = niceliksel tehdit

- Kök: saldırganın **hedefi** · Dallar: ona ulaşma yolları.
- **VEYA**: en ucuz çocuk yeter · **VE**: çocukların toplamı gerekir.
- Yapraklara **maliyet** yazarsak en ucuz saldırıyı buluruz.

HEDEF (VEYA)

├ Veritabanını çöz	(VE: $3 + 6 = 9$)
├ Bellekten oku	(VEYA: min 2) <- en ucuz
└ Trafigi dinle	(VE: $7 + 8 = 15$)

Demo 04 — Savunma maliyeti artırır

code/week-02/04-saldiri-agaci

```
ONCE:  EN UCUZ SALDIRI = 2  (bellekten oku)
SONRA: EN UCUZ SALDIRI = 8  (RASP eklendi)
```

- "Bellekten oku" dalına **RASP** (anti-debug, anti-hook) eklendi.
- En ucuz saldırı **2 → 8** birime çıktı; en zayıf nokta **değişti**.
- Güvenlik yatırımını **sayıyla** gerekçelendirmenin yolu budur.

Denetim kaydı: ne yazılır, ne yazılmaz?

✓ Kaydet	✗ Asla kaydetme
Giriş denemeleri (kim, ne zaman, nereden)	Parola, PIN, belirteç, özel anahtar
Yetki ve ayar değişiklikleri	Kart numarasının tamamı, kişisel veri
Hassas işlemler (onay, dışa aktarma, silme)	Çözülmüş gizli veri
Güvenlik denetimi sonuçları	Sürümde hata ayıklama çıktısı

Tarif 13.11 · günlüğü **saldırganın da okuyacağını** varsayarak tasarla

Demo 10 — Günlük enjeksiyonu (CWE-117)

code/week-02/10-gunluk-enjeksiyonu

GUVENSIZ:

```
DENETIM giris kullanıcı=kayra
DENETIM giris kullanıcı=admin sonuc=BASARILI ... <- SAHTE satir
DENETIM giris kullanıcı=deniz^[2K^Mgizli ... <- ekran silme
```

GUVENLI:

```
DENETIM giris kullanıcı=kayra\x0A2026-... (tek satir)
sahte 'admin BASARILI' satiri: 0
```

✓ Kaçışla (`\xNN`) · ✓ uzunluğu sınırla · ✓ `syslog(LOG_INFO, "%s", girdi)` — girdi asla biçim dizgesi değil

Demo 11 — Kurcalamaya dayanıklı günlük

code/week-02/11-kurcalamaya-dayanikli-gunluk · HMAC zinciri + **anahtar evrimi**

Saldırı	Anahtar evrimli	Statik anahtar
Kayıt değiştirme	✓ MAC:BOZUK	✓ yakalanır
Kayıt silme / sıra değiştirme	✓ SIRA:BOZUK	✓ yakalanır
Ele geçirilen anahtarla geçmişi yeniden yazma	✓ KURCALANMIŞ	✗ BÜTÜNLÜKLÜ görünür

Anahtar her kayıta tek yönlü evrilir, eski anahtar silinir → ele geçirmeden **önceki** kayıtlar korunur

Erişim denetimi ve modeller

Özne, nesne, hak — ve matris

- **Özne** (kullanıcı/süreç) · **Nesne** (dosya/kayıt) · **Hak** (oku/yaz/çalıştır).

	odeme_anahitari	gunluk
uygulama	—	yaz
kutuphane	oku, yaz	yaz
saldirgan	—	—

Matris iki biçimde saklanır: **ACL** (sütun: nesne kimin erişeceğini tutar) · **Yetenek** (satır: özne biletleri).

DAC · MAC · RBAC

Model	Kararı kim verir?	Örnek
DAC	Nesnenin sahibi	Unix <code>chmod</code> , Windows DACL
MAC	Sistem/politika (sahip değiştiremez)	Bell–LaPadula, SELinux
RBAC	İzinler rollere , kullanıcı role atanır	"Muhasebeci" rolü

Gerçek sistemler üçünü **birlikte** kullanır; etkin karar çoğu zaman **kesişimdir**.

Demo 03 — Salt DAC (matris)

```
code/week-02/03-erisim-modeli
```

```
kutuphane OKU odeme_anahtari => IZIN
uygulama OKU odeme_anahtari => RED (matriste yok)
saldirgan OKU odeme_anahtari => RED
saldirgan YAZ gunluk          => RED
```

Uygulamanın bile ödeme anahtarına **doğrudan erişimi yok** → **en az ayrıcalık**.

Bell–LaPadula: gizlilik

Seviler: Genel < Gizli < Çok Gizli

- **Yukarı okuma yok:** kendinden gizli olanı okuyamaz.
- **Aşağı yazma yok (*-özellik):** kendinden az gizli olana yazamaz.

"Read down, write up" — gizli bilgi **aşağı sızamaz.**

Biba: bütünlük (BLP'nin tersi)

Seviyeler: Dış < Uygulama < Çekirdek (güvenilirlik)

- **Aşağı okuma yok:** kendinden az güvenilir veriyi okuyamaz.
- **Yukarı yazma yok:** kendinden güvenilir veriye yazamaz.

"Read up, write down" — kirli bilgi **yukarı çıkamaz**, temizi bozamaz.

BLP ve Biba: ters yönler

BELL-LaPADULA (gizlilik)

Cok Gizli	^	oku
		yaz gonderme
Gizli		
	v	
Genel		

BIBA (butunluk)

Cekirdek	^	yaz
		oku gonderme
Uygulama		
	v	
Dis		

BLP: yukari OKUMA yok, asagi YAZMA yok

Biba: asagi OKUMA yok, yukari YAZMA yok

Bir veri hem gizli hem güvenilir olmalıysa → yalnız **kendi seviyesinde** işlenir.

BLP ↔ Biba — şema

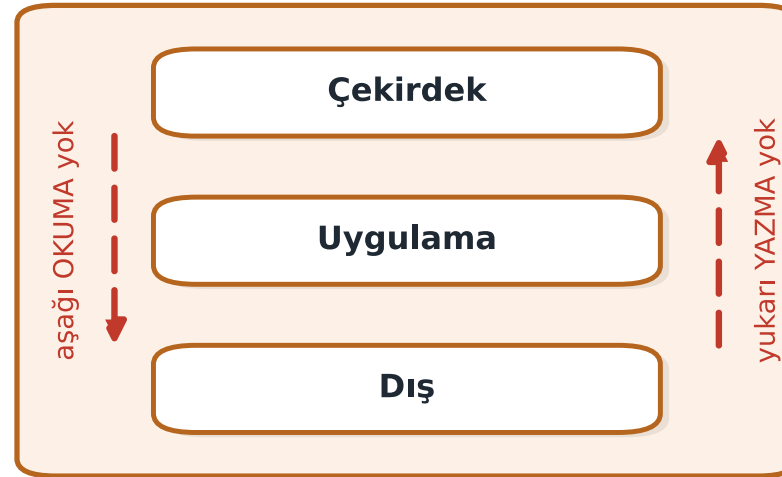
Bell-LaPadula ↔ Biba: ters yönler

BLP gizliliği, Biba bütünlüğü korur

Bell-LaPadula (GİZLİLİK)



Biba (BÜTÜNLÜK)



Aynı kafes, ters kurallar: biri sırrın aşağı sızmasını, diğeri çöpün yukarı çıkmasını engeller.

Ters yönler: BLP gizliliği (yukarı okuma + aşağı yazma yok), Biba bütünlüğü (aşağı okuma + yukarı yazma yok) korur.

Clark–Wilson: ticari bütünlük

Askeri seviye yerine **iyi biçimli işlem + görev ayrılığı** (1987, banka):

- **CDI**: bütünlüğü korunacak veri (bakiye).
- **TP**: CDI'ye yalnız **onaylı işlemler** dokunur (doğrudan yazma yok).
- **IVP**: tutarlılığı düzenli denetler ("borç = alacak").
- **Görev ayrılığı**: başlatan $\neq$ onaylayan.

"Veriye doğrudan değil, **doğrulanmış işlemlerle** dokun."

Demo 03 — BLP ve Biba çıktısı

BLP: memur OKU operasyon	BLP:RED => RED	(yukari okuma)
general YAZ ilan	BLP:RED => RED	(asagi yazma)
Biba: aglayici YAZ kayıt	BIBA:RED => RED	(yukari yazma)
islemci OKU gelen	BIBA:RED => RED	(asagi okuma)

DAC her ikisinde **VAR**; reddi getiren **sistem (MAC)** kuralıdır.

Unix erişim modeli

`-rwxr-x---` → sahip / grup / diğerleri, her biri **oku-yaz-çalıştır**.

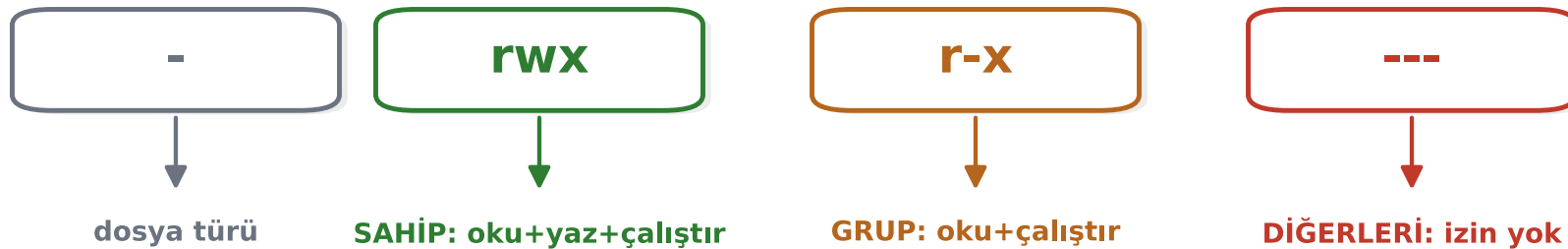
- Üç kimlik: **effective / real / saved** UID.
- **setuid**: program **sahibinin** (çoğu zaman root) yetkisiyle çalışır — güçlü ama tehlikeli.
- **sticky bit**: `/tmp` 'te başkasının dosyasını silememe.

Kitap: Tarif 2.1

Unix izin bitleri — şema

Unix izin bitleri: -rwxr-x---

üç grup, üçer bit



Her grup 3 bit: r=4, w=2, x=1 → rwx = 7, r-x = 5, --- = 0 → chmod 750

En az ayrıcalık: 'diğerleri' sütununu boş bırakmak çoğu sızıntıyı baştan keser.

Windows erişim modeli

Kavram	Unix	Windows
Kimlik	UID/GID	SID
İzin	Mod bitleri	DACL içindeki ACE'ler
Denetim	syslog (harici)	SACL (yerleşik)
"Herkes açık"	chmod 777	NULL DACL

- **DACL**: kim ne yapabilir (ACE = SID + hak + izin/reddet).
- Modern Windows: **MIC** ve **AppContainer** (Biba benzeri). Kitap: Tarif 2.2

Demo 06 — TOCTOU yarış (CWE-367)

code/week-02/06-toctou (yalnız WSL/Linux) · Kitap: Tarif 2.3

```
GUVENSIZ (once lstat, sonra fopen):  
  >>> SALDIRI BASARILI: yazi gizli_hedef.txt'e yonlendirildi!  
GUVENLI (O_NOFOLLOW ile tek adim):  
  reddedildi: hedef sembolik bag
```

Denetim ile kullanım arasındaki boşluk bir yarışdır. Çözüm: tek atomik adım (`O_NOFOLLOW` , `O_EXCL`), fd üzerinden çalış.

Demo 12 — ACE değerlendirme algoritması

code/week-02/12-ace-degerlendirme · Tarif 2.2

```
1a kanonik:  RET Herkes TAM | IZIN Pazarlama YAZ => RED
1b bozuk:    IZIN Pazarlama YAZ | ... RET Herkes => IZIN
2a NULL DACL (yabanci hesap) => IZIN (SAHIP_AL, IZIN_YAZ dahil!)
2b BOS DACL  (ayni hesap)    => RED
3b kalitim kesildi           => RED (liste bitti)
```

- Sıra önemli: **açık RET** → **açık İZİN** → **miras RET** → **miras İZİN**
- **✗** NULL DACL = herkese **her** hak · **✓** gerekli hakları açıkça ver
- Sahip, DACL ne derse desin izinleri değiştirebilir

Demo 13 — Unix izinleri, umask, setuid

code/week-02/13-unix-izin-umask · Tarif 2.1, 2.7, 1.3

```
ayse oku paradoks.txt ---rwxrwx sahip -> RED
zeynep oku paradoks.txt ---rwxrwx diger -> IZIN
0666 & ~umask: 000 -> rw-rw-rw- 022 -> rw-r--r-- 077 -> rw-----
seteuid(1001) r/e/s = 1001/1001/0 <- sakli UID hala 0!
```

- **İlk eşleşen sınıf** karar verir (sahip → grup → diğerleri)
- `fopen()` 0666 ister → hassas dosya: `open(..., 0600)`
- Kalıcı bırakma: `setresuid()` + doğrula

Demo 14 — RBAC, görev ayrılığı, Clark–Wilson

code/week-02/14-rbac-clark-wilson · banka benzetimi

```
ATA cem GISE          SSD(GISE,ONAYCI)  -> RED
OTURUM deniz GISE,DENETCI  DSD(GISE,DENETCI) -> RED
OTURUM deniz DENETCI      -> TAMAM
deniz IVP: beklenen 20250, bulunan 20250
```

- **SSD**: çelişen iki role atanamaz · **DSD**: aynı oturumda ikisi etkin olamaz
- CDI'ya yalnız **sertifikalı TP** dokunur; büyük havale → **ikinci kişi onayı**
- ⚠ Hatalı TP sertifikalanırsa kurallar görmez, **IVP yakalar**

Zafiyet sınıflandırma

CWE: zayıflık türleri kataloğu

- **CWE** = zayıflığın **türü** (SQL enjeksiyonu → CWE-89).
- **CWE Top 25**: o yıl en çok görülen/tehlikeli 25 zayıflık.

CWE	Zayıflık
CWE-787	Sınır dışına yazma
CWE-79	XSS
CWE-89	SQL enjeksiyonu
CWE-367	TOCTOU (Demo 06)
CWE-798	Gömülü kimlik/anahtar

Hiyerarşik: pillar → class → base → variant. **En somut** CWE'yi seç.

OWASP Top 10 ve MASVS

Belge	Kapsam	Türü
CWE / Top 25	Tüm yazılım	Katalog / öncelik
OWASP Top 10	Web	Farkındalık / öncelik
OWASP ASVS	Web	Doğrulama standardı
OWASP MASVS + MASTG	Mobil	Doğrulama + test

MASVS-RESILIENCE = tersine mühendisliğe direnç → bu dersin kod gizleme (H9, H14), RASP (H6), whitebox (H11) konuları.

CVE ve CVSS

- **CVE:** belirli ürünlerdeki belirli açığın **kimliği** (CVE-2014-0160 = Heartbleed). Bir **isim**, ciddiyet değil.
- **CVSS:** açığın **ciddiyeti** 0.0–10.0. Üç grup: **Temel** (değişmez), Zamansal, Çevresel.

Bir CVE bir ya da birden çok **CWE türüne** aittir.

CVSS v3.1 Temel puan metrikleri

Metrik	Sorusu	Değerler
AV	Nereden?	Ağ / Bitişik / Yerel / Fiziksel
AC	Ne kadar zor?	Düşük / Yüksek
PR	Yetki gerekir mi?	Yok / Düşük / Yüksek
UI	Kurban gerekir mi?	Yok / Gerekli
S	Kapsam aşıyor mu?	Değişmedi / Değişti
C/I/A	Etki?	Yüksek / Düşük / Yok

v4.0: kapsam yerine ayrı VC/VI/VA + SC/SI/SA; temel puan = CVSS-B.

Demo 05 — CVSS hesaplayıcı

code/week-02/05-cvss (Python)

```
AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H = 9.8 Kritik
... S:U -> S:C = 10.0 Kritik
AV:L (yere1) /PR:L/... C:H/I:H/A:H = 7.8 Yüksek
```

- **Uzaktan + kimliksiz** en yüksek; kapsam değişince 10.0.
- ⚠ Temel puan **bağlamı bilmez**: 4.0'lık açık tam sizin varlığını vuruyorsa sizin için 9.8'den önemli olabilir.

Zafiyet yaşam döngüsü

Kesif -> Özel ifsa -> Yama -> Yama yayini -> Kamuya acilma -> Guncelleme
\\..... zero-day (yama yokken saldiri)

- **Zero-day:** satıcının bilmediği/yamasının olmadığı açık.
- **Sorumlu ifşa:** önce satıcıya bildir, ~90 gün bekle, sonra açıkla.
- **Yama boşluğu:** yama çıkınca açık kamuya açılır; güncellemeyen savunmasız (WannaCry).

Dönem projesi: bu hafta (S4)

- [] S4 saldırı/tehdit tablonuzdaki **her satıra bir CWE** ekleyin.
- [] En kritik varlığa bir **saldırı ağacı** (Demo 04 biçimi); en ucuz yol + en etkili önlem.
- [] Kullandığınız bileşenlerin **CVE**'lerini bulup **CVSS** puanlayın.
- [] Uygun **erişim modelini** (DAC / MAC / RBAC) gerekçelendirin.

ID	Tehdit	Varlık	Yol	CWE	Önlem	Bölüm
----	--------	--------	-----	-----	-------	-------

Kendini sinama (seçme)

1. Virüs, solucan, truva atı farkı?
2. Polimorfizm $\neq$ metamorfizm: fark?
3. Yüksek entropi zararlıyı **kanıtlar** mı?
4. BLP iki kuralı ve neyi korur?
5. Biba, BLP'den nasıl ayrılır?
6. DAC ile MAC farkı?
7. CWE ile CVE farkı?
8. CVSS Temel puan neyi ölçmez?

Kendini sınaama — cevaplar (1–4)

1. **Virüs** başka programa bulaşır; **solucan** kendi başına ağdan kopyalar; **truva atı** faydalı görünüp gizli zararlı taşır.
2. **Polimorfik** her kopyada kendini farklı **şifreler**; **metamorfik** kodunu **yeniden yazar**.
3. **Hayır**. Yüksek entropi bir **ipucudur** (şifreli/paketli); tek başına zararlıyı kanıtlamaz (meşru sıkıştırma da yüksek entropili).
4. **BLP**: yukarı okuma yok + aşağı yazma yok → **gizliliği** korur.

Kendini sınaama — cevaplar (5–8)

5. **Biba** BLP'nin tersidir: aşağı okuma yok + yukarı yazma yok → **bütünlüğü** korur.
6. **DAC** izinleri sahibi belirler (esnek, sızıntıya açık); **MAC** sistemin zorunlu etiketli kuralları (katı).
7. **CWE** zayıflık **türü** (ör. CWE-416); **CVE** belirli bir ürünlerdeki **somut** açık.
8. **CVSS Temel puan** olabilirliği/etkiyi ölçer ama **istismarın sahada gerçekleşme olasılığını** (tehdit/ortam) tek başına ölçmez.

Gelecek hafta

Hafta 3 — Veri Güvenliği: Aktarımda, Beklemede, Kullanımda

Şifreleme temelleri · TLS 1.3 ve sertifika sabitleme · AES-GCM ile yerel şifreleme · güvenli silme

Kaynak: Viega & Messier, Tarif 9.1–9.3, 10.7–10.9, 4.9–4.11, 13.2

Ek · Erişim modelleri karşılaştırma

BLP · gizlilik kuralı

- "Yukarı yazma, aşağı okuma yok" değil — tersi:
- **Yukarı okuma yok** (no read up): düşük yetkili, yüksek gizli veriyi okuyamaz.
- **Aşağı yazma yok** (no write down): yüksek yetkili, düşük seviyeye sızdıramaz.

BLP · örnek

- "Gizli" etiketli bir kullanıcı "Çok Gizli" belgeyi **okuyamaz**.
- "Çok Gizli" bir süreç "Genel" dosyaya **yazamaz** (sızıntı önlenir).

Amaç: **gizlilik**.

Biba · bütünlük kuralı

- BLP'nin **tersi**:
- **Aşağı okuma yok**: yüksek bütünlüklü, düşük güvenilir veriyi okumaz.
- **Yukarı yazma yok**: düşük güvenilir, yüksek bütünlüğe yazamaz.

Amaç: **bütünlük**.

BLP vs Biba

	BLP	Biba
Korur	Gizlilik	Bütünlük
Okuma	Yukarı yok	Aşağı yok
Yazma	Aşağı yok	Yukarı yok

Aynı sistemde ikisi **çelişebilir**; dikkatle birleştirilir.

Clark–Wilson

- Ticari bütünlük: iyi biçimli işlemler + görev ayrımı.
- Veriye yalnız **onaylı** yordamlarla erişilir.
- Örnek: muhasebe kayıtları.

DAC / MAC / RBAC · seçim

- **DAC:** esnek ama sızıntıya açık (sahibi paylaşır).
- **MAC:** katı, askeri/etiketli ortam.
- **RBAC:** kurumsal; rol bazlı, yönetimi kolay.

Ek · Zafiyeti bul (kod okuma)

Örnek 1 · günlük enjeksiyonu

```
fprintf(log, "giris: %s\n", kullanıcı_adi); /* CWE-117 */
```

Kullanıcı adı `\n` içerirse **sahte** günlük satırı ekler. Doğrusu: kaçış/temizleme.

Örnek 2 · biçim dizisi

```
syslog(LOG_INFO, kullanıcı_girdisi); /* CWE-134 */
```

Biçim dizgesi sabit olmalı: `syslog(LOG_INFO, "%s", girdi)`.

Örnek 3 · zayıf tespit

- Yalnız **imza** ile tarama, polimorfik virüsü kaçıırır.
- Ek: sezgisel + davranış + entropi.

Ek · Çözümlü kendini sinama

Soru 1

Virüs ile solucan farkı?

Cevap: Virüs başka programa bulaşarak yayılır; solucan kendi başına ağ üzerinden kendini kopyalar.

Soru 2

Polimorfik virüs neden imza tespitini zorlar?

Cevap: Her kopyada kendini farklı şifreler; sabit bir imza kalmaz. Sezgisel/emülasyon gerekir.

Soru 3

İmza ve sezgisel tespitin güçlü/zayıf yanı?

Cevap: İmza hızlı ama yeniyi kaçıırır; sezgisel yeniyi bulur ama yanlış alarm üretir.

Soru 4

BLP ile Biba neyi korur, kuralları ters mi?

Cevap: BLP gizliliği (yukarı okuma yok), Biba bütünlüğü (aşağı okuma yok). Evet, yönleri terstir.

Soru 5

CWE ile CVE farkı?

Cevap: CWE zayıflık **türü** (örn. use-after-free); CVE belirli bir ürünlerdeki **somut** açık.

Soru 6

CVSS ne söyler?

Cevap: Bir açığın ciddiyetini standart bir puanla (0–10); önceliklendirmede kullanılır.

Soru 7

Günlük enjeksiyonu (CWE-117) nasıl önlenir?

Cevap: Günlüğe yazılan kullanıcı verisindeki satır sonu/kontrol karakterlerini temizle/kaçır; yapılandırılmış günlük kullan.

Soru 8

Sorumlu ifşa nedir?

Cevap: Açığı önce üreticiye bildirip yama için süre tanımak, sonra kamuya açıklamak.

Sözlük

Terim	Anlam
Polimorfik	Her kopyada farklı şifreli
İmza/sezgisel	Bilinen iz / şüpheli davranış
Entropi	Rastgelelik ölçüsü
DAC/MAC/RBAC	Erişim denetim modelleri
BLP/Biba	Gizlilik / bütünlük modeli
CWE/CVE/CVSS	Tür / somut açık / puan

Son söz (2. hafta)

Tehdidi **sınıflandır**, erişimi **modelle**, açığı **kataloglar**la (CWE/CVE) izle ve ciddiyetini (CVSS) ölç.

Bu çerçeve, sonraki haftalardaki teknik korumaların zeminidir.